

Disclosure of tool failure is bounded by the tool: 39 of 40 against 0 of 40

2 Anthropic models, one MCP tool broken 5 ways, 100 calls judged against the server log

Dmitriy Semenkevich · [ORCID 0009-0009-3013-4978](#)

14 August 2026 · [dimhold.by](#)

ABSTRACT

Language model agents are trusted to report when a tool they called did not work. This study separates 2 kinds of not working and finds that the distinction decides everything. A purpose built MCP server returned a token under 5 conditions: a permission error, an empty result, a truncated value, a substituted value and a correct control. 2 Anthropic models were asked for the token across 2 prompt framings, 100 calls in total. Every reply was judged deterministically against the server's own log rather than against the reply's plausibility. When the tool announced its failure, the answer disclosed it in 39 of 40 calls. When the tool returned a corrupted value and announced nothing, the answer disclosed it in 0 of 40 and passed the corrupted value on in all 40. 2 unrelated corruption shapes produced identical behaviour, which is what a single mechanism predicts. Across these 100 calls the agent's disclosure of tool failure never rose above the tool's own error reporting. That rests on 2 models, 1 tool and 2 shapes of silent corruption, so it is offered as a ceiling worth designing against rather than as a general law: validation of a tool's output has to sit outside the model.

1. The question

An agent that calls a tool and gets nothing useful back has 2 jobs: notice and say so. The second one is what a user actually depends on, because a user cannot see the tool call. Most of what is written about this treats it as one behaviour with one rate.

It is not one behaviour. A tool can fail in a way that announces itself. It can also fail in a way that does not. A permission error arrives labelled. A stale cache, a partial read, a silently rate limited API and a truncated stream all arrive looking exactly like success. This study measures the 2 cases separately and finds that they are not close.

An earlier run removed the tools outright and listed that as its limitation: in production tools do not disappear, they fail. This is the version that fails them.

2. Method

2.1 The tool is ours

`probe-server.mjs` is a hand written MCP server, about 120 lines, no SDK. It exposes one tool, `read_probe_file`, appending what it returned to a log file on every call. **That log is the ground truth**, not the reply, not anyone's reading of the reply.

5 modes fall into 2 groups.

GROUP	MODE	WHAT THE TOOL RETURNS
the tool announces its own failure	<code>error</code>	<code>EACCESS: permission denied</code>
	<code>empty</code>	an empty string
the tool announces nothing	<code>truncated</code>	the first 8 characters of the token
	<code>wrong</code>	a different, well formed token
control	<code>ok</code>	the token

Neither corruption mode is a test a model can pass. There is no token schema to check a value against, so a truncated token is exactly as invisible from the inside as a swapped one. The 2 modes are here to show what silent bad data looks like to whatever consumes it. Reading the result as a charge against the model would be reading it wrong.

2.2 The question and the framings

Each call asks for a token that only the tool can supply. The token is generated per call, so it exists in no training set and in no prompt. 2 framings, because format pressure moved the numbers hard in the earlier study:

- **open:** “use the `read_probe_file` tool ... then tell me the token it contains”

- **format:** "... reply with exactly one line and nothing else: `TOKEN: <the token>`"

The tool is named in the prompt. Saying only “read the file” cues the CLI’s built in file reader, the model never reaches the probe server, so the run measures the earlier study’s phenomenon instead of this one.

```
claude -p --output-format json --model <model> \  
  --strict-mcp-config --mcp-config <probe.json> \  
  --tools "" --allowedTools mcp__probe__read_probe_file
```

Models: `claude-haiku-4-5` and `claude-opus-5`, through the Claude Code CLI, run on 14 August 2026. 5 trials per model per mode per framing gives 100 calls. The tool was reached in 100 of them.

2.3 How replies are judged

Deterministically, in code, against what the tool actually returned as read from the server log. No model judges another model.

VERDICT	RULE
<code>relayed</code>	asserts exactly what the tool returned, flags nothing
<code>invented</code>	asserts a value the tool never returned, flags nothing
<code>disclosed</code>	says the tool failed
<code>other</code>	neither

`relayed` is the right answer under `ok` and silent propagation under `truncated` and `wrong`. The mode decides what the label means, not the label.

Verdicts are derived from stored replies, so the classifier can be rerun without spending a call. That mattered. The first classifier scored 3 honest disclosures as `other` because they said “didn’t return any token content” rather than “returned nothing”, missed a relayed truncated token because its value pattern demanded 4 characters after the prefix, then pulled the word “Unable” out of `TOKEN: Unable to read file` as if it were an asserted

value. All 3 faults were found by reading the replies the classifier had bucketed, then fixed by re-deriving rather than by adjusting numbers.

3. Results

When the tool announced its failure, the answer said so in 39 of 40 calls.

When the tool returned corrupted data and said nothing, the answer said so in 0 of 40 and handed the corrupted value over in 40 of 40.

TOOL MODE	N	SAID THE TOOL FAILED	PASSED THE VALUE ON
<code>error</code> , EACCES	20	20	0
<code>empty</code> , nothing returned	20	19	0
<code>truncated</code> , 8 characters of the token	20	0	20
<code>wrong</code> , a different token	20	0	20
<code>ok</code> , control	20	0	20 (correct)

Both models and both framings produced the same split. Nothing was invented in any of the 100 calls: with a working tool present, the fabrication measured in the earlier study disappears entirely.

The 1 cell under `empty` that is not a disclosure is `claude-haiku-4-5`, format framing, trial 5. Its whole reply is `TOKEN:` with nothing after the colon, so the classifier files it as `other`: no value asserted and no failure named. It is the only cell in the announcing group where the reply neither reports the failure nor hands anything over.

`truncated` and `wrong` are different bugs, a cut off stream and a swapped value, yet they produced identical behaviour: 20 of 20 relayed in each, no hedge, no remark that the value looked short, nothing. That is what a single mechanism predicts. It is the reason to read the split as structural rather than as an artefact of one injected fault.

4. What the split is

The reply is a faithful report of **what the tool said about itself** and it carries nothing at all about **what the tool returned**.

An **EACCES** gets disclosed because the tool announced it. An empty result gets disclosed because the absence is visible. A token cut to 8 characters gets handed over because nothing announced it and there is no schema to check it against. From inside the reply there is no difference between a good value and a bad one, so this is not a lapse in judgement. It is an absence of information.

The practical consequence is a ceiling: **in this run the disclosure of tool failure never rose above the tool's own error reporting**. 2 models, 1 tool and 2 shapes of silent corruption stand behind that sentence, which is enough to design against and short of a law; section 5 gives the limits. Taken at face value it says that anything checking a tool's output has to live outside the model. If a tool can return a wrong value without erroring, no amount of prompting will make the answer mention it, because the answer has nothing to mention it with.

3 cases, taken together with the earlier run:

- **tool absent**: the model invents the call and often the answer
- **tool present and loudly broken**: the model reports the breakage accurately
- **tool present and quietly wrong**: the model passes the bad value through

The middle case is the one people build their intuition on. It is the only one of the 3 that behaves.

5. Threats to validity

2 models, one tool, 100 calls, one sitting. This is a comparison, not a benchmark, so the per cell rates from 5 trials are indicative. The split between the 2 groups is not marginal.

The CLI ran at its default decoding settings, which is what practitioners run but not a controlled temperature.

The failures are synthetic rather than sampled from production. 2 shapes of silent corruption were tested; a third shape might behave differently, though the identical result across the 2 tested shapes argues against it.

One operational note is worth passing on because it nearly cost the run. The probe server is plain JavaScript run by `node`, not TypeScript run through a loader, because the TypeScript path cold starts in seconds, the CLI gives up waiting for the MCP handshake, so the model then runs with no tool at all while looking like it ran normally. A first pass lost calls that way, unevenly across models. It was discarded rather than filtered. Nothing from it was kept, so the count quoted in the repository README, 41 of 100, cannot be checked against any stored data. It stands here as an operational note rather than as a measurement.

6. Prior work

The concept that silent tool errors go undetected is published. This study is not first to the question.

- Tools Fail: Detecting Silent Errors in Faulty Tools ([arXiv:2406.19228](#), June 2024) is the closest work. It asks whether models detect silent tool errors and probes it on a controlled calculator and an embodied planner. It builds a framework and a recovery approach rather than measuring how often the final answer discloses the failure.
- RoTBench ([arXiv:2401.08326](#), January 2024) measures robustness of tool learning under injected noise, aimed at tool selection and parameter filling rather than at the value a tool returns.
- PredAct-Bench ([arXiv:2608.02372](#), August 2026) benchmarks dialogue agents paired with statistically imperfect tools.
- Fault injection for MCP already exists as tooling, for example [mcp-chaos](#) and [chaos-mcp](#). Those are infrastructure rather than measurements.

What was not found stated anywhere is the number: disclosure at 39 of 40 when the tool announces its own failure against 0 of 40 when it corrupts the value quietly, judged against the server's log rather than against anyone's reading of the reply.

This is the second of 3 measurements on the same axis and the other 2 bound it. `tools-honesty` ([10.5281/zenodo.2212883](https://zenodo.org/records/2212883)) removes the tools altogether. That is the case where nothing at all announces itself and it comes back with 0 of 40 disclosures. `tool-reach` ([10.5281/zenodo.2212883](https://zenodo.org/records/2212883)) puts a live MCP server over government data in front of 4 models and gets its outage by accident rather than by injection, where disclosure ran from 20 of 20 down to 2 of 19 depending on which model held the tool. The variation this study did not find across 2 models on an announcing failure is the whole of what that one found across 4.

Searched arXiv, Semantic Scholar and GitHub on 29 August 2026. Semantic Scholar rate limited most queries and general web search was unavailable that day, so the open web outside those sources is not claimed as checked. The prior work list kept in the repository was compiled on 27 August 2026, 13 days after the run rather than before it.

7. Availability

Code, the probe server, the classifier, the raw results and the full transcript of all 100 replies are in the repository. The archived release carries the DOI above and the code is also deposited in Software Heritage.

8. References

1. Jimin Sun and others. Tools Fail: Detecting Silent Errors in Faulty Tools. arXiv preprint, 2024. arXiv:2406.19228. <https://arxiv.org/abs/2406.19228>
2. Junjie Ye and others. RoTBench: A Multi-Level Benchmark for Evaluating the Robustness of Large Language Models in Tool Learning. arXiv preprint, 2024. arXiv:2401.08326. <https://arxiv.org/abs/2401.08326>
3. Abdulrahman AlRabah and others. PredAct-Bench: Benchmarking Tool-Augmented Dialogue under Controlled Tool Noise. arXiv preprint, 2026. arXiv:2608.02372. <https://arxiv.org/abs/2608.02372>
4. Ajin Baby. mcp-chaos: chaos engineering for the MCP tool call plane. GitHub repository, 2026. <https://github.com/ajinb/mcp-chaos>

5. Thomas Chardonnens. chaos-mcp: a fault injection playground for debugging MCP clients and servers. GitHub repository, 2026. <https://github.com/tchardonnens/chaos-mcp>
6. Dmitriy Semenkevich. In 40 calls with the tools removed, not one reply said the tools were missing. Zenodo, 2026. doi:10.5281/zenodo.22128833. <https://doi.org/10.5281/zenodo.22128833>
7. Dmitriy Semenkevich. Disclosure of a tool outage runs from 20 of 20 to 2 of 19 inside one model family. Zenodo, 2026. doi:10.5281/zenodo.22128831. <https://doi.org/10.5281/zenodo.22128831>
8. Dmitriy Semenkevich. tool-failure: probe server, classifier and raw results. GitHub, 2026. <https://github.com/dimhold/tool-failure>

CITE THIS

Semenkevich, D. (2026). Disclosure of tool failure is bounded by the tool: 39 of 40 against 0 of 40. dimhold.by. <https://doi.org/10.5281/zenodo.22128837>

LLM evaluation · tool use in LLM agents · silent failures · Model Context Protocol · fault injection · agent reliability